Richard Smith
2017-11-07

P0588R1: Simplifying implicit lambda capture

Also:
Core Issue 1632: Lambda capture in member initializers
Core Issue 1913: `decltype((x))` in *lambda-expression*s
(Core issue?): effect of structured bindings on lambda capture unspecified

Change in 6 basic paragraph 3:

> An *entity* is a value, object, reference, **structured binding**, function, enumerator, type, class member, bit-field, template, template specialization, namespace, or parameter pack.

Add a new paragraph after 6 basic paragraph 6 ("A *variable* is ..."):

> A *local entity* is a variable with automatic storage duration ([basic.stc.auto]), a structured binding ([dcl.struct.bind]) whose corresponding variable is such an entity, or the `*this` object ([expr.prim.this]).

Change in 6.2 basic.def.odr paragraph 3, splitting into paragraphs as indicated:

> A variable x whose name appears as a potentially-evaluated expression ex is odr-used by ex unless applying the lvalue-to-rvalue conversion (7.1) to x yields a constant expression (8.20) that does not invoke any non-trivial functions and, if x is an object, ex is an element of the set of potential results of an expression e, where either the lvalue-to-rvalue conversion (7.1) is applied to e, or e is a discarded-value expression (Clause 8).
>
> **A structured binding is odr-used if it appears as a potentially-evaluated expression.**
>
> `*this` is odr-used if ~~`it`~~ **`this`** appears as a potentially-evaluated expression (including as the result of the implicit transformation in the body of a non-static member function (12.2.2)).
>
> A virtual member function is odr-used if it is not pure. A function whose name appears as a potentially-evaluated expression is odr-used if it is the unique lookup result or the selected member of a set of overloaded functions ...
>
> An allocation or deallocation function for a class is odr-used by a *new-expression* ...
>
> An assignment operator function in a class is odr-used by an implicitly-defined copy assignment or move-assignment function for another class as specified in 15.8. A constructor for a class is odr-used as specified in 11.6. A destructor for a class is odr-used if it is potentially invoked (15.4).

Drafting note: we may wish to further constrain what qualifies as an odr-use of a structured binding when we start to allow structured binding declarations to be declared `constexpr`.

Add a new paragraph before 6.2 basic.def.odr paragraph 4 ("Every program shall contain exactly one definition"):

> A local entity ([basic]) is *odr-usable* in a declarative region ([basic.scope.declarative]) if:
>
> - the local entity is either not `*this`, or an enclosing class or non-lambda function parameter scope exists and, if the innermost such scope is a function parameter scope, it corresponds to a non-static member function, and
> - for each intervening declarative region ([basic.scope.declarative]) between the point at which the entity is introduced (where `*this` is considered to be introduced within the innermost enclosing class or non-lambda function definition scope) and the region, either:
>   - the declarative region is a block scope, or
>   - the declarative region is the function parameter scope of a *lambda-expression* that has a *simple-capture* naming the entity or has a *capture-default*.
>
> If a local entity is odr-used in a declarative region in which it is not odr-usable, the program is ill-formed. [*Example*:
>
> ```
> void f(int n) {
>   [] { n = 1; };         // error, n is not odr-usable due to intervening lambda-expression
>   struct A {
>     void f() { n = 2; }  // error, n is not odr-usable due to intervening function definition scope
>   };
>   void g(int = n);       // error, n is not odr-usable due to intervening function prototype scope
>   [&] { [n]{ n = 3; } }; // ok
> }
> ```
>
>]

Add a new paragraph after 6.3.1 basic.scope.declarative paragraph 4 ("Given a set of declarations in a single declarative region"):

> For a given declarative region *R* and a point *P* outside *R*, the set of *intervening* declarative regions between *P* and *R* comprises all declarative regions that are or enclose *R* and do not enclose *P*.

Move 6.3.3 basic.scope.block paragraph 2 to a new subclause 6.3.4 basic.scope.param ("Function parameter scope"), with the following changes:

A function parameter (including one appearing in a *lambda-declarator*) or function-local predefined variable ([dcl.fct.def]) has *function parameter scope*. The potential scope of a ~~function~~ parameter ~~name~~ ~~(including one appearing in a lambda-declarator)~~ or of a function-local predefined variable ~~in a function definition (11.4)~~ begins at its point of declaration. **If the nearest enclosing function declarator is not the declarator of a function definition, the potential scope ends at the end of that function declarator. Otherwise, if** ~~If~~ the function has a function-try-block the potential scope ~~of a parameter or of a function-local predefined variable~~ ends at the end of the last associated handler **~~, otherwise~~ . Otherwise the potential scope** ~~it~~ ends at the end of the outermost block of the function definition. A parameter name shall not be redeclared in the outermost block of the function definition nor in the outermost block of any handler associated with a function-try-block.

Delete 6.3.4 basic.scope.proto ("Function prototype scope").

Change in 8.1.4.1 expr.prim.id.unqual paragraph 1, inserting a paragraph break as shown:

[...] ~~The type of the expression is the type of the *identifier*.~~

The result is the entity denoted by the identifier. **If the entity is a local entity and naming it from outside of an unevaluated operand within the declarative region where the *unqualified-id* appears would result in some intervening *lambda-expression* capturing it by copy, the type of the expression is the type of a class member access expression ([expr.ref]) naming the non-static data member that would be declared for such a capture in the closure object of the innermost such intervening *lambda-expression*. [*Note*: If that *lambda-expression* is not declared `mutable`, the type of such an identifier will typically be `const` qualified.] Otherwise, the type of the expression is the type of the result. [*Note*: The type will be adjusted as described in Clause [expr] if it is *cv*-qualified or is a reference type.]** The expression is an lvalue if the entity is a function, variable, or data member and a prvalue otherwise; it is a bit-field if the identifier designates a bit-field.

[*Example*:

```
void f() {
  float x, &r = x;
  [=] {
    decltype(x) y1; // y1 has type float
    decltype((x)) y2 = y1; // y2 has type float const& because this lambda is not mutable and x is an lvalue
    decltype(r) r1 = y1; // r1 has type float&
    decltype((r)) r2 = y2; // r2 has type float const&
  };
}
```

]

Drafting note: the above example is moved from 8.1.5.2 expr.prim.lambda.capture paragraph 14 with minor changes.

Change in 8.1.5.2 expr.prim.lambda.capture paragraph 3:

A *lambda-expression* ~~whose smallest enclosing scope is a block scope (6.3.3)~~ is a *local lambda expression* **if its innermost enclosing scope is a block scope ([basic.scope.block]), or if it appears within a default member initializer and its innermost enclosing scope is the corresponding class scope ([basic.scope.class])**; any other *lambda-expression* shall not have a *capture-default* or *simple-capture* in its *lambda-introducer*. ~~The *reaching scope* of a local lambda expression is the set of enclosing scopes up to and including the innermost enclosing function and its parameters. [*Note*: This reaching scope includes any intervening *lambda-expressions*.]~~

Change in 8.1.5.2 expr.prim.lambda.capture paragraph 4:

The identifier in a *simple-capture* is looked up using the usual rules for unqualified name lookup (6.4.1); each such lookup shall find ~~an~~ **a local** entity. **The *simple-captures* `this` and `* this` denote the local entity `*this`.** An entity that is designated by a *simple-capture* is said to be *explicitly captured*~~, and shall be *this (when the simple-capture is â€œthisâ€ or â€œ* thisâ€) or a variable with automatic storage duration declared in the reaching scope of the local lambda expression~~.

Change in 8.1.5.2 expr.prim.lambda.capture paragraph 7:

For the purposes of lambda capture, an expression potentially references local entities as follows:

- **An *id-expression* that names a local entity potentially references that entity; an *id-expression* that names one or more non-static class members and does not form a pointer to member ([expr.unary.op]) potentially references `*this`. [*Note*: This occurs even if overload resolution selects a static member function for the *id-expression*.]**
- **A `this` expression potentially references `*this`.**
- **A *lambda-expression* potentially references the local entities named by its *simple-captures*.**

If an expression potentially references a local entity within a declarative region in which it is odr-usable, and the expression would be potentially evaluated if the effect of any enclosing `typeid` expressions ([expr.typeid]) were ignored, the entity is said to be *implicitly captured* by each intervening ~~A~~ *lambda-expression* with an associated *capture-default* that does not explicitly capture **it.** ~~*this or a variable with automatic storage duration (this excludes any id-expression that has been found to refer to an init-captureâ€™s associated non-static data member), is said to *implicitly capture* the entity (i.e., *this or a variable) if the *compound-statement*:~~

- ~~odr-uses (6.2) the entity (in the case of a variable),~~
- ~~odr-uses (6.2) this (in the case of the object designated by *this), or~~
- ~~names the entity in a potentially-evaluated expression (6.2) where the enclosing full-expression depends on a generic lambda~~

~~parameter declared within the reaching scope of the lambda-expression.~~

[*Example*:

```
void f(int, const int (&)[2] = {}); ~~{ }~~   // #1
void f(const int&, const int (&)[1]); ~~{ }~~ // #2
void test() {
  const int x = 17;
  auto g = [](auto a) {
    f(x); // OK: calls #1, does not capture x
  };


  **auto g1 = [=](auto a) {**
    **f(x); // OK: calls #1, captures x**
  **};**


  auto g2 = [=](auto a) {
    int selector[sizeof(a) == 1 ? 1 : 2]{};
    f(x, selector); // OK: ~~is a dependent expression, so~~ captures x**, might call #1 or #2**
  };


  **auto g3 = [=](auto a) {**
    **typeid(a + x);  // captures x regardless of whether a + x is an unevaluated operand**
  **};**

}
```

Within `g1`, an implementation might optimize away the capture of `x` as it is not odr-used.] [*Note*: The set of captured entities is determined syntactically, and entities might be implicitly captured even if the expression denoting a local entity is within a discarded statement ([stmt.if]). [*Example*:

```
template<bool B>
void f(int n) {
  [=](auto a) {
    if constexpr (B && sizeof(a) > 4) {
      (void)n;
    }
  }(0); // captures n regardless of the value of B and sizeof(int)
}
```

]] ~~All such implicitly captured entities shall be declared within the reaching scope of the lambda expression. [*Note*: The implicit capture of an entity by a nested lambda-expression can cause its implicit capture by the containing lambda-expression (see below). Implicit odr-uses of this can result in implicit capture.]~~

Change in 8.1.5.2 expr.prim.lambda.capture paragraph 8:

An entity is captured if it is captured explicitly or implicitly. An entity captured by a lambda-expression is odr-used (6.2) in the scope containing the lambda-expression. ~~If *this is captured by a local lambda expression, its nearest enclosing function shall be a non-static member function. If a lambda-expression or an instantiation of the function call operator template of a generic lambda odr-uses (6.2) this or a variable with automatic storage duration from its reaching scope, that entity shall be captured by the lambda-expression.~~ If a *lambda-expression* **explicitly** captures an entity ~~and~~ that ~~entity~~ is not **odr-usable** ~~defined or captured in the immediately enclosing lambda expression or function~~ **or captures a structured binding (explicitly or implicitly)**, the program is ill-formed. [*Example*:

```
…
auto m4 = [&,j] {    // error: j not odr-usable due to intervening lambda ~~not captured by~~ m3
  int x = n;         // error: n is odr-used but not odr-usable due to intervening lambda ~~implicitly captured by m4 but not captured by~~ m3
  x += m;            // OK: …
  x += i;            // error: i is ~~outside of the reaching scope~~ odr-used but not odr-usable due to intervening function and class scopes
…
]
```

Drafting note: the wording for structured bindings here is a placeholder, to be replaced by whatever behavior we actually want for lambda capture of structured bindings.

Change in 8.1.5.2 expr.prim.lambda.capture paragraph 11:

Every id-expression within the compound-statement of a lambda-expression that is an odr-use (6.2) of an entity captured by copy is transformed into an access to the corresponding unnamed data member of the closure type. [Note: An id-expression that is not an odr-use refers to the original entity, never to a member of the closure type. ~~Furthermore~~ **However**, such an id-expression ~~does not~~ **can still** cause the implicit capture of the entity.] If `*this` is captured by copy, each **expression that** odr-use**s** ~~of~~ `*this` is transformed ~~into a pointer to~~**to instead refer to** the corresponding unnamed data member of the closure type~~, cast (8.4) to the type of this~~. ~~[Note: The cast ensures that the transformed expression is a prvalue. â€" end note]~~ […]

Delete 8.1.5.2 expr.prim.lambda.capture paragraph 14:

~~Every occurrence of `decltype((x))` where `x` is a possibly parenthesized *id-expression* that names an entity of automatic storage duration is treated as if `x` were transformed into an access to a correspnding data member of the closure type that would have been declared if `x` were an odr-use of the denoted entity. [*Example*: …]~~

Change in 11.3.5 dcl.fct paragraph 14:

> [...] ([basic.scope.~~proto~~**param**]) [...]

Change in 11.3.6 dcl.fct.default paragraph 7:

> **[*Note*:** A local variable ~~shall not appear as a potentially-evaluated expression~~ **cannot be odr-used ([basic.def.odr])** in a default argument. **]** [...]

Change in 12.4 class.local paragraph 1:

> **[*Note*: A declaration** ~~Declarations~~ in a local class ~~shall~~ **can**not odr-use (6.2) ~~a variable with automatic storage duration~~ **a local entity** from an enclosing scope. **]** [*Example*:
>
> …
> ```
> int arr[1, 2];
> auto [y, z] = arr;
>
> struct local {
>   int g() { return x; }    // error: odr-use of automatic non-odr-usable variable x
>   …
>   int* n() { return &N; }  // error: odr-use of automatic non-odr-usable variable N
>   int p() { return y; }    // error: odr-use of non-odr-usable structured binding y
> };
> …
> ```
>
>]

Change in 17.8.1 temp.inst paragraph 9:

> An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, a member class, a static data member of a class template, or a substatement of a constexpr if statement (9.4.1), unless such instantiation is required. **[*Note*: The instantiation of a generic lambda does not require instantiation of substatements of a constexpr if statement within its *compound-statement* unless the call operator template is instantiated.]** [...]

Add entry to Annex C:

> [expr.prim.lambda.capture]
> **Change:** Implicit lambda capture may capture additional entities.
> **Rationale:** Rule simplification, necessary to resolve interactions with constexpr if.
> **Effect on original feature:** Lambdas with a *capture-default* may capture local entities that were not captured in C++ 2017 if those entities are only referenced in contexts that do not result in an odr-use.